0022. Generate Parentheses

Medium | Category: Stack | LeetCode: https://leetcode.com/problems/generate-pa

Problem

- Return all well-formed strings of n pairs of parentheses.
- Example: n = 3 →
  ["((()))","(()())","(())()","()(())","()()()"].

Intuition

- Build left to right. We may add "(" while open < n, and ")" only while close < open.
- Those two rules guarantee every completed string is valid, so nothing is generated and then thrown away.

Brute Force

- Generate all 2^(2n) strings and validate each: O(2^(2n) n).

Optimal Approach

- Backtrack(sb, open, close).
- If length == 2n, record.
- If open < n: append "(", recurse, remove.
- If close < open: append ")", recurse, remove.

Worked Example

```
n = 2.

""   -> "("   -> "((" -> "(()" -> "(())"  record
             -> "()" -> "()(" -> "()()"  record
```

Java Solution

```java
public class GenerateParentheses {
    public List<String> generateParenthesis(int n) {
        List<String> res = new ArrayList<>();
        go(new StringBuilder(), 0, 0, n, res);
        return res;
    }

    private void go(StringBuilder sb, int open, int close, int n, List<String> res) {
        if (sb.length() == 2 * n) { res.add(sb.toString()); return; }
        if (open < n) { sb.append('('); go(sb, open + 1, close, n, res); sb.deleteCharAt(sb.length() - 1); }
        if (close < open) { sb.append(')'); go(sb, open, close + 1, n, res); sb.deleteCharAt(sb.length() - 1); }
    }
}
```

Complexity

- Time: O(4^n / sqrt(n)) (the Catalan number of results, each of length 2n). Space: O(n) recursion.

Pitfalls

- Undo the append (backtrack) after each branch when using a shared StringBuilder.